



PR - DE LA TEORÍA A LA PRÁCTICA: WALKTHROUGH EN ACCIÓN

Programacion Wap

Mtro. Servando López Contreras

David Campos Morado – 183511
Carrera: Ingeniería en Tecnologías de la Información

Contents

Objetivo de la actividad	2
Descripción del escenario	2
Configuración del laboratorio	2
Evidencias y descripción del funcionamiento.....	3
Reconocimiento y Descubrimiento de Red	3
Enumeración Web con Gobuster	3
Intercepción de Tráfico y Modificación con Burp Suite.....	3
Explotación del Registro Forzado (API REST)	4
Post-Explotación y Extracción de Secretos	4
Explicación del flujo de captura de información	4
Controles de seguridad propuestos.....	5
Conclusión.....	5

Objetivo de la actividad

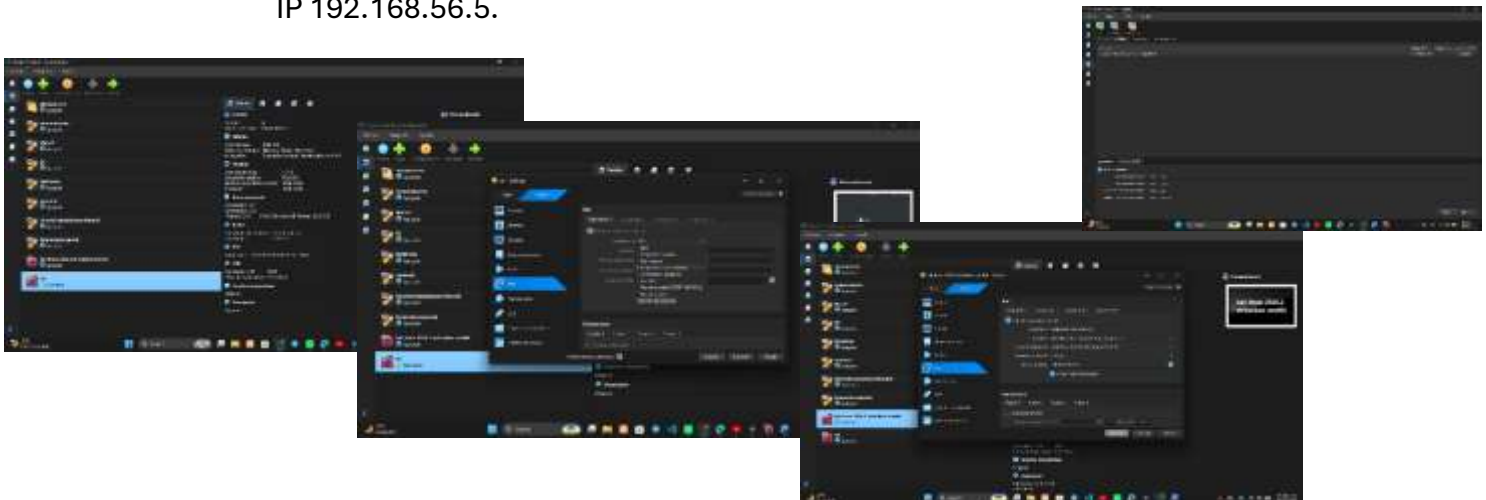
Demostrar la aplicación práctica de los conceptos teóricos de seguridad informática mediante un *walkthrough* técnico detallado. Se evalúa un entorno controlado de pruebas para identificar vulnerabilidades en los servicios web, manipular peticiones de una API REST mediante interceptación de proxy y documentar los vectores de ataque orientados a la extracción de información sensible y escalada de privilegios.

Descripción del escenario

El ejercicio se centra en el análisis de seguridad de la máquina virtual "Snakeoil" en un entorno local aislado. Tras realizar un reconocimiento de red y escaneo de puertos, se detectaron servicios expuestos en los puertos 22, 80 y 8080. Al descartar el ataque tradicional por fuerza bruta sobre un hash PBKDF2 debido a su alta complejidad computacional, se pivotó hacia la auditoría de la API web expuesta en el puerto 8080. A través de la alteración de verbos HTTP y la elusión de restricciones en los *endpoints*, se logró el registro de un usuario no autorizado, la obtención de tokens de sesión y la visualización de configuraciones internas del backend.

Configuración del laboratorio

- **Entorno de virtualización:** Oracle VirtualBox con aislamiento de red mediante adaptador de tipo "Solo anfitrión" (Host-Only).
- **Servidor DHCP:** Habilitado en la red virtual para la asignación dinámica de direcciones IP del segmento 192.168.56.0/24.
- **Máquina atacante:** Kali Linux, operando desde las interfaces de red configuradas localmente (192.168.56.3 y 192.168.56.4).
- **Máquina víctima:** Máquina virtual vulnerable "Snakeoil" identificada en la dirección IP 192.168.56.5.



Evidencias y descripción del funcionamiento

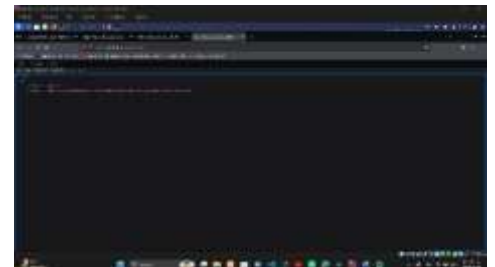
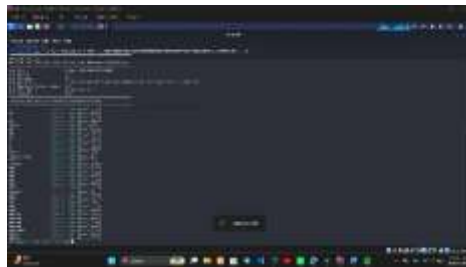
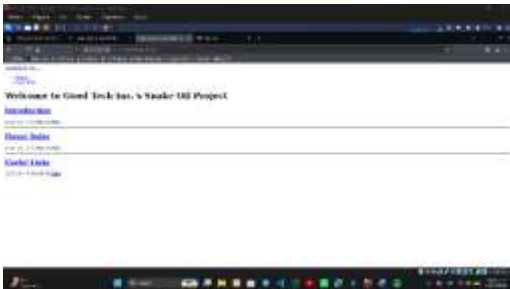
Reconocimiento y Descubrimiento de Red

Se empleó la herramienta netdiscover para escanear el segmento de red activo, localizando la IP del objetivo (192.168.56.5). Posteriormente, un escaneo con Nmap (nmap -sC -sV) confirmó la disponibilidad de OpenSSH y los servicios Nginx en los puertos 80 y 8080.



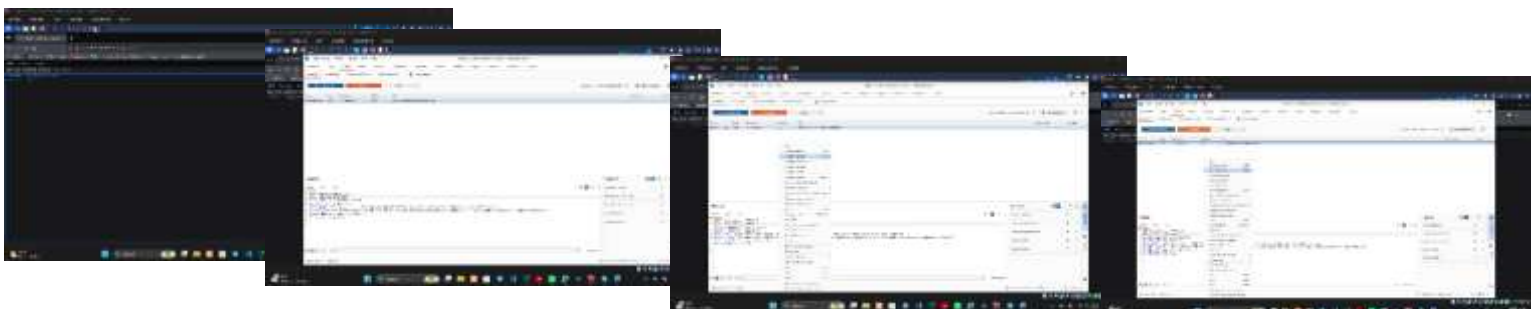
Enumeración Web con Gobuster

Mediante un escaneo de directorios con Gobuster, se mapearon los *endpoints* principales de la aplicación en el puerto 8080, destacando rutas críticas como /users, /registration, /login, /secret y /run. La ruta /users expuso un hash PBKDF2 del usuario "patrick", cuya inversión por diccionario resultó inviable por limitaciones de hardware.



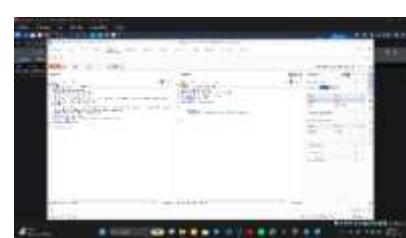
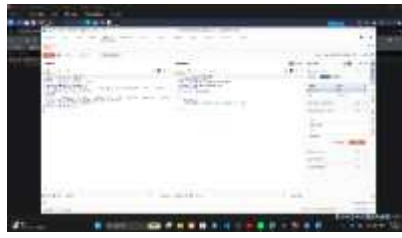
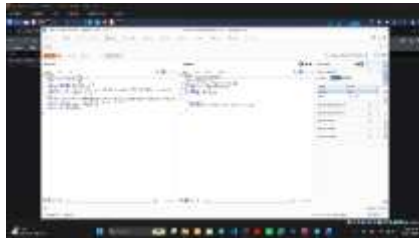
Intercepción de Tráfico y Modificación con Burp Suite

Ante las restricciones de acceso web convencional (errores tipo "Wrong Method!!!!" al intentar consultar rutas por método GET), se implementó **Burp Suite** como proxy de intercepción. Al capturar la petición hacia /registration, se envió al *Repeater* para modificar el verbo HTTP de GET a POST.



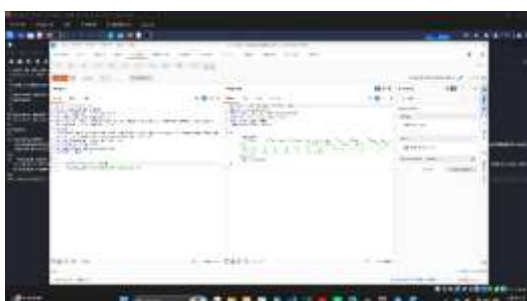
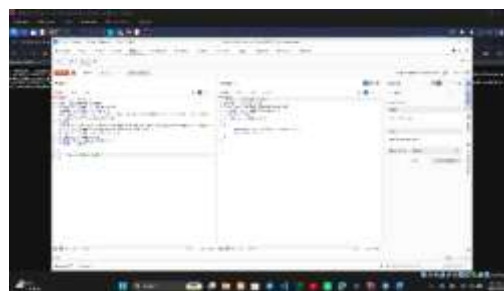
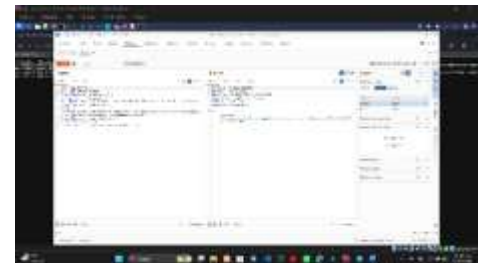
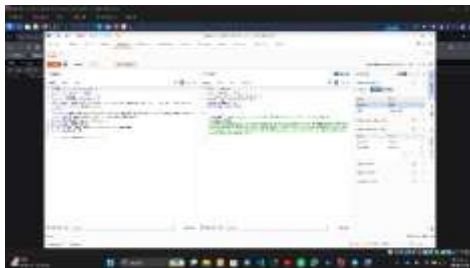
Explotación del Registro Forzado (API REST)

Al enviar la petición POST, el servidor solicitó de manera interactiva los campos faltantes (username y password). Configurando el formato como URL-encoded, se inyectó un *payload* con las credenciales deseadas (username=morado&password=morado4), lo que provocó una respuesta exitosa (200 OK) con la creación del usuario y la emisión de tokens JWT de acceso (access_token y refresh_token).



Post-Explotación y Extracción de Secretos

Utilizando los tokens y rutas como /login y /run, se evaluaron las capacidades de ejecución y lectura del backend. Mediante el envío de parámetros JSON específicos, se forzó al servidor a exponer fragmentos del código fuente principal (app.py), revelando configuraciones críticas de Flask y claves de cifrado en texto plano.



```
app = Flask(__name__)
api = Api(app)
app.config['SECRET_KEY'] = 'snakeoilisnetgoodforcorporations'
app.config['JWT_COOKIE_SECURE'] = True
app.config['JWT_SECRET_KEY'] = 'N0reasonab1e00UBTth1sP4SSW0RD1sG00D'
app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(minutes=15)
app.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(hours=1)
app.config['JWT_TOKEN_LOCATION'] = ['cookies']
app.config['JWT_COOKIE_CSRF_PROTECT'] = False # development setting!

app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
```

Explicación del flujo de captura de información

- **Manipulación de métodos HTTP:** Las APIs REST implementan restricciones basadas en verbos. Forzar métodos incorrectos o no esperados (POST en lugar de GET) permite evaluar la sanitización y el enrutamiento de controladores desprotegidos.
- **Interceptación de tráfico (Proxy):** Burp Suite permite inspeccionar y alterar los datos en tránsito, rompiendo las validaciones estáticas implementadas únicamente en la interfaz de usuario del navegador.
- **Tokens JWT (JSON Web Tokens):** Mecanismo de autenticación basado en estados distribuidos. La obtención de un token válido concede privilegios de sesión sin requerir almacenamiento local persistente en el servidor.

Controles de seguridad propuestos

- **Control de Acceso Basado en Roles (RBAC):** Restringir el acceso a los *endpoints* sensibles de registro y administración, impidiendo la creación libre de cuentas con privilegios.
- **Sanitización y Validación de Entradas:** Implementar filtros robustos en el backend para evitar la alteración maliciosa de verbos y parámetros HTTP.
- **Protección de Secretos y Configuración:** Almacenar claves maestras (SECRET_KEY) y credenciales en variables de entorno seguras en lugar de mantenerlas *hardcodeadas* en el código fuente.

Conclusión

La práctica demostró que cuando los métodos tradicionales de fuerza bruta criptográfica fallan debido a la robustez de los algoritmos (como PBKDF2), la auditoría lógica de las aplicaciones web y sus APIs ofrece rutas de vulnerabilidad alternativas. La correcta manipulación de peticiones mediante herramientas como Burp Suite evidenció cómo fallas de diseño en el enrutamiento y la validación de métodos HTTP pueden comprometer la integridad y confidencialidad de un sistema completo.